

HW6 – Harder Problems Only (Corrected)

Matched to the attached HW6 question PDF.

1. 64-bit / 32-bit memory chip built from two $2^k \times 32$ chips

Let Chip 0 store the low 32 bits and Chip 1 store the high 32 bits when the big chip is used in 64-bit mode.

(a) What feeds the two chips' Addr and DataIn inputs?

In both modes, the lower k address bits are exactly A , so both chips receive the same address bus:

$$\text{Chip0.Addr} = A, \quad \text{Chip1.Addr} = A.$$

In 64-bit mode ($S_1 = 1$), Chip 0 gets $I_{31:0}$ and Chip 1 gets $I_{63:32}$. In 32-bit mode ($S_1 = 0$), the selected 32-bit word is addressed by S_0A , so both chips may still receive A , but only one chip is enabled for writing; the 32-bit write data comes from $I_{31:0}$. Thus:

$$\begin{aligned} \text{Chip0.DataIn} &= I_{31:0}, \\ \text{Chip1.DataIn} &= \begin{cases} I_{31:0}, & S_1 = 0, \\ I_{63:32}, & S_1 = 1. \end{cases} \end{aligned}$$

Equivalently,

$$\text{Chip1.DataIn} = \text{MUX}(S_1, I_{31:0}, I_{63:32}).$$

(b) Expressions for W_i

In 64-bit mode ($S_1 = 1$), both chips must be written. In 32-bit mode ($S_1 = 0$), $S_0 = 0$ selects Chip 0 and $S_0 = 1$ selects Chip 1. Therefore:

$$W_0 = W(S_1 + \overline{S_0}) = WS_1 + W\overline{S_0},$$

$$W_1 = W(S_1 + S_0) = WS_1 + WS_0.$$

These are the correct write-enable signals.

(c) Circuitry for the 64-bit output

In 64-bit mode ($S_1 = 1$):

$$O_{63:32} = \text{Chip1.Out}, \quad O_{31:0} = \text{Chip0.Out}.$$

In 32-bit mode ($S_1 = 0$):

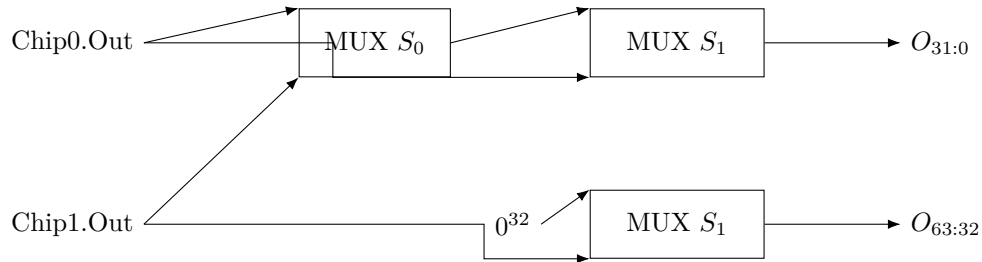
$$O_{63:32} = 0,$$

$$O_{31:0} = \begin{cases} \text{Chip0.Out}, & S_0 = 0, \\ \text{Chip1.Out}, & S_0 = 1. \end{cases}$$

So a clean implementation is:

$$O_{63:32} = \text{MUX}(S_1, 0, \text{Chip1.Out}),$$

$$O_{31:0} = \text{MUX}(S_1, \text{MUX}(S_0, \text{Chip0.Out}, \text{Chip1.Out}), \text{Chip0.Out}).$$



2. Memory with reset

The question states that when **reset**=1, memory should be cleared.

(a) Clear the entire memory when **reset**=1

To clear all words, every row must be selected and the data written must be all zeros. A correct modification is:

- replace each decoder output d_i by $d_i \vee \text{reset}$,
- replace the data bus by $\text{DATA} \wedge \overline{\text{reset}}$ bitwise (equivalently, a MUX that outputs 0^k when **reset**=1 and outputs DATA when **reset**=0).

Then:

- if **reset**=0, the circuit behaves normally;
- if **reset**=1, all rows are enabled and the data forced into memory is 0^k .

(b) Clear only addresses less than the specified address

The warmup 1MASK circuit produces a 2^m -bit vector M such that $M_i = 1$ iff the address value is greater than i . That is exactly the set of rows with addresses less than the given address.

So during reset, the normal decoder outputs should be ignored and the row-selection vector should come from the 1MASK block instead. The correct per-row control is:

$$\text{row}_i = (\overline{\text{reset}} \wedge d_i) \vee (\text{reset} \wedge M_i).$$

The data bus is still forced to zero during reset:

$$\text{DataToMemory} = \text{DATA} \wedge \overline{\text{reset}}.$$

This exactly matches the requirement: normal addressing when $\text{reset}=0$, and clearing all rows with address $< A$ when $\text{reset}=1$.

(c) What contiguous ranges can be cleared with coincident selection?

With coincident selection, the address is split into row bits (m_1 bits) and column bits ($m_0 = m - m_1$ bits). A reset operation selects some set of rows and some set of columns, so the cells cleared are the Cartesian product

$$\text{SelectedRows} \times \text{SelectedColumns}.$$

Therefore the cleared region must be a rectangle in the row/column view of memory.

So a contiguous interval in linear address order can be cleared only if one of the following holds:

1. the entire interval lies inside a single row, or
2. the interval starts at column 0 of its first row, ends at column $2^{m_0} - 1$ of its last row, and contains every full row in between.

Hence coincident selection cannot in general clear an arbitrary interval $[A, B]$ that starts in the middle of one row and ends in the middle of a later row.

3. Register / MUX / Shift / Adder circuit

From the actual wiring in the diagram:

- S_0 controls the two left MUXes, so $S_0 = 0$ makes both registers hold their current values, and $S_0 = 1$ loads both registers from the final bus.
- The middle upper MUX selects A when $S_1 = 0$ and B when $S_1 = 1$; its output then goes into the shifter.
- The shifter shifts left when $S_1 = 0$ and shifts right when $S_1 = 1$.
- The MUX after the shifter selects A when $S_1 = 0$ and the shift output when $S_1 = 1$.
- The lower middle MUX selects B when $S_1 = 0$ and DATA when $S_1 = 1$; that value is the bottom adder input.
- The adder uses top input A , and computes $A + B$ when $S_1 = 0$ and $A - \text{DATA}$ when $S_1 = 1$.
- The final rightmost MUX selects the non-adder path when $S_2 = 0$ and the adder path when $S_2 = 1$.

(a) All 8 combinations of $S_2S_1S_0$

$S_2S_1S_0$	Final bus value	Effect
000	A	hold both registers
001	A	load both registers with A
010	$\text{shr}(B)$	hold both registers
011	$\text{shr}(B)$	load both registers with $\text{shr}(B)$
100	$A + B$	hold both registers
101	$A + B$	load both registers with $A + B$
110	$A - \text{DATA}$	hold both registers
111	$A - \text{DATA}$	load both registers with $A - \text{DATA}$

Here $\text{shr}(B)$ means “shift B right by one bit,” because when $S_1 = 1$ the upper-middle MUX chooses B and the shifter is in right-shift mode.

(b) All settings that transfer the value in A into B

To copy the current value of A into B , the circuit must:

- load the registers, so $S_0 = 1$ is required, and
- make the final bus equal to the current value of A .

There are two ways to do that from the given wiring:

Method 1: direct non-adder path

$$S_2 = 0, \quad S_1 = 0, \quad S_0 = 1,$$

with DATA arbitrary. Then the final bus is just A , so both registers load A and in particular $B \leftarrow A$.

Method 2: adder path with DATA set to 0

$$S_2 = 1, \quad S_1 = 1, \quad S_0 = 1, \quad \text{DATA} = 0.$$

Then the final bus is

$$A - \text{DATA} = A - 0 = A,$$

so again $B \leftarrow A$.

So the complete set of selector/data choices that always perform the transfer is:

$$(S_2, S_1, S_0) = (0, 0, 1) \text{ with } \text{DATA} \text{ arbitrary,}$$

$$(S_2, S_1, S_0) = (1, 1, 1) \text{ with } \text{DATA} = 0.$$